



K.R. MANGALAM UNIVERSITY

**Database Management Lab File
(ENCS205)**

**Bachelor of Technology
in
Computer Science and Engineering**

**Submitted by
Aditya Bhatia (2401010030)**

B. Tech CSE (Core) – 4 Semester

**Submitted To:
MS. Mansi Kajal**

School of Engineering & Technology

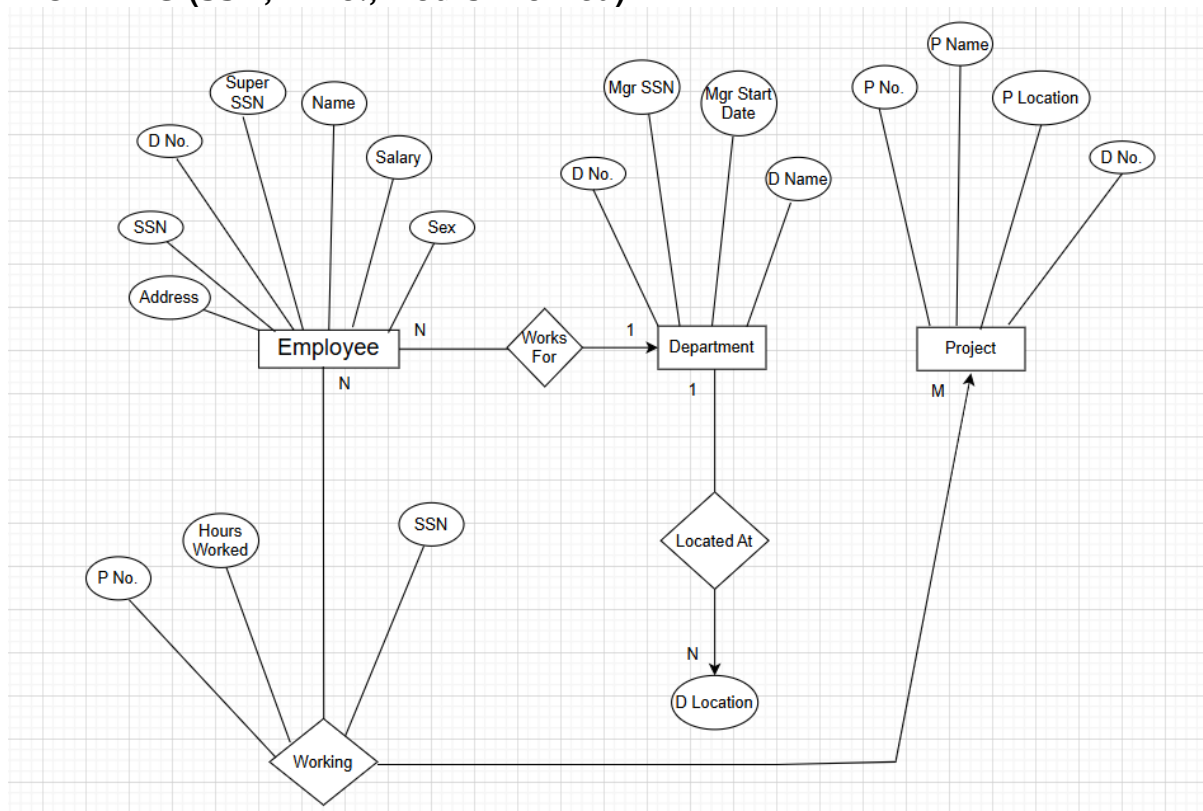
**K. R. MANGALAM UNIVERSITY Sohna, Haryana
122103, India Jan-2026**

EXPERIMENT-1

1. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:

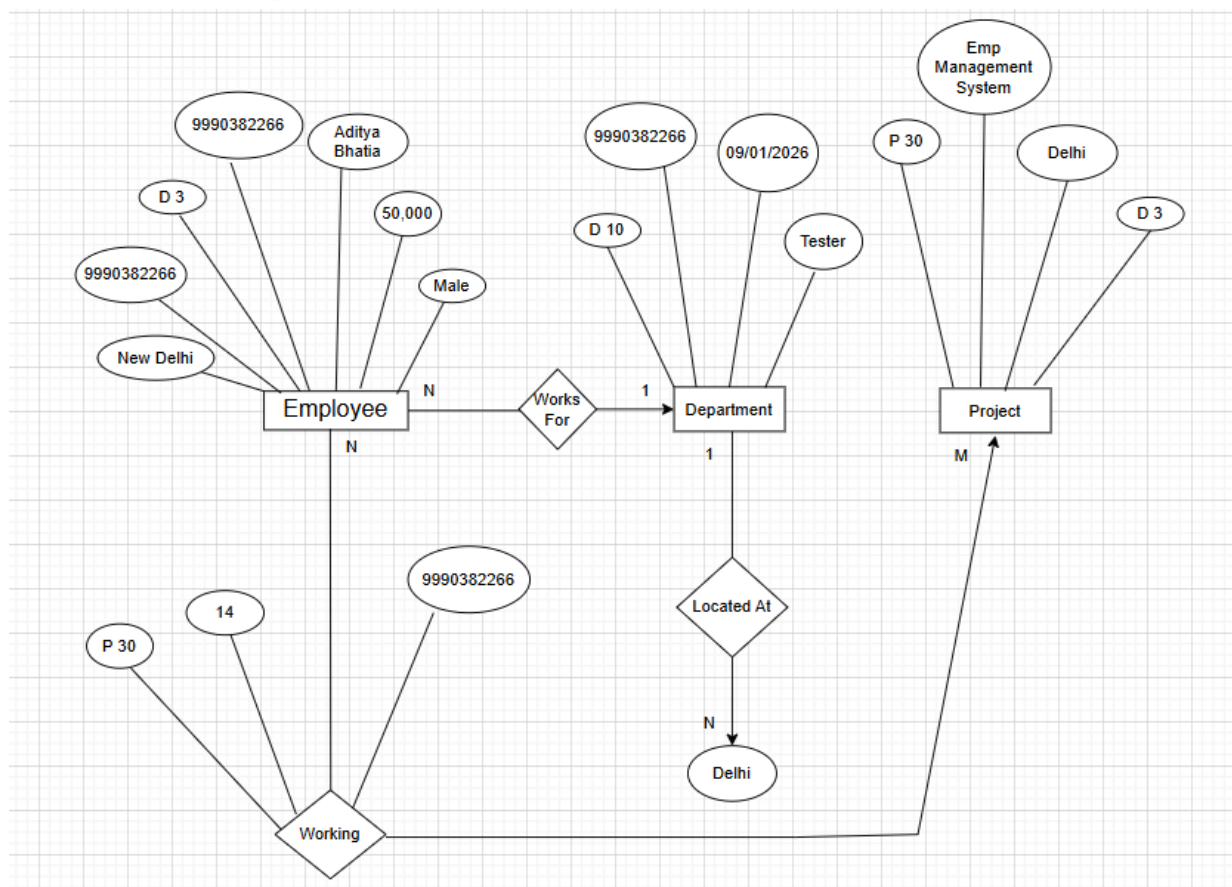
EMPLOYEE (SSN, Name, Address, Sex, Salary, Super SSN, D No.)
DEPARTMENT (D No., D Name, Mgr SSN, Mgr Start Date)
DLOCATION (D Location)
PROJECT (P No., P Name, P Location, D No.)
WORKING (SSN, P No., Hours Worked)



Explanation of ER Diagram – Company Database :

- This ER diagram represents a company database that stores information about employees, departments, locations, and projects.
- The Employee entity stores details such as SSN, name, address, sex, salary, supervisor SSN, and department number. Each employee works for a department. This relationship is shown using WORKS FOR. Many employees can work for one department, but each employee works for only one department.
- The Department entity stores department information such as department number, department name, manager SSN, and manager start date. A department can have multiple locations. This is shown using the LOCATED AT relationship between Department and D Location.

- The Project entity stores information about projects such as project number, project name, project location, and department number. Each project is controlled by one department, and one department can control many projects.
- Employees work on different projects. This many-to-many relationship is shown using the WORKS ON relationship. The number of hours an employee works on a project is stored as an attribute of this relationship.



Company ER Diagram Explained Using Real Example :

The ER diagram consists of three main entities: Employee, Department, and Project. Employees work for departments through the Works For relationship (N:1). Departments are located at specific places through the Located At relationship. Departments manage multiple projects (1:M). Employees work on projects through the Working relationship, which stores additional attributes such as working hours, forming an M:N relationship.

EXPERIMENT-2

2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

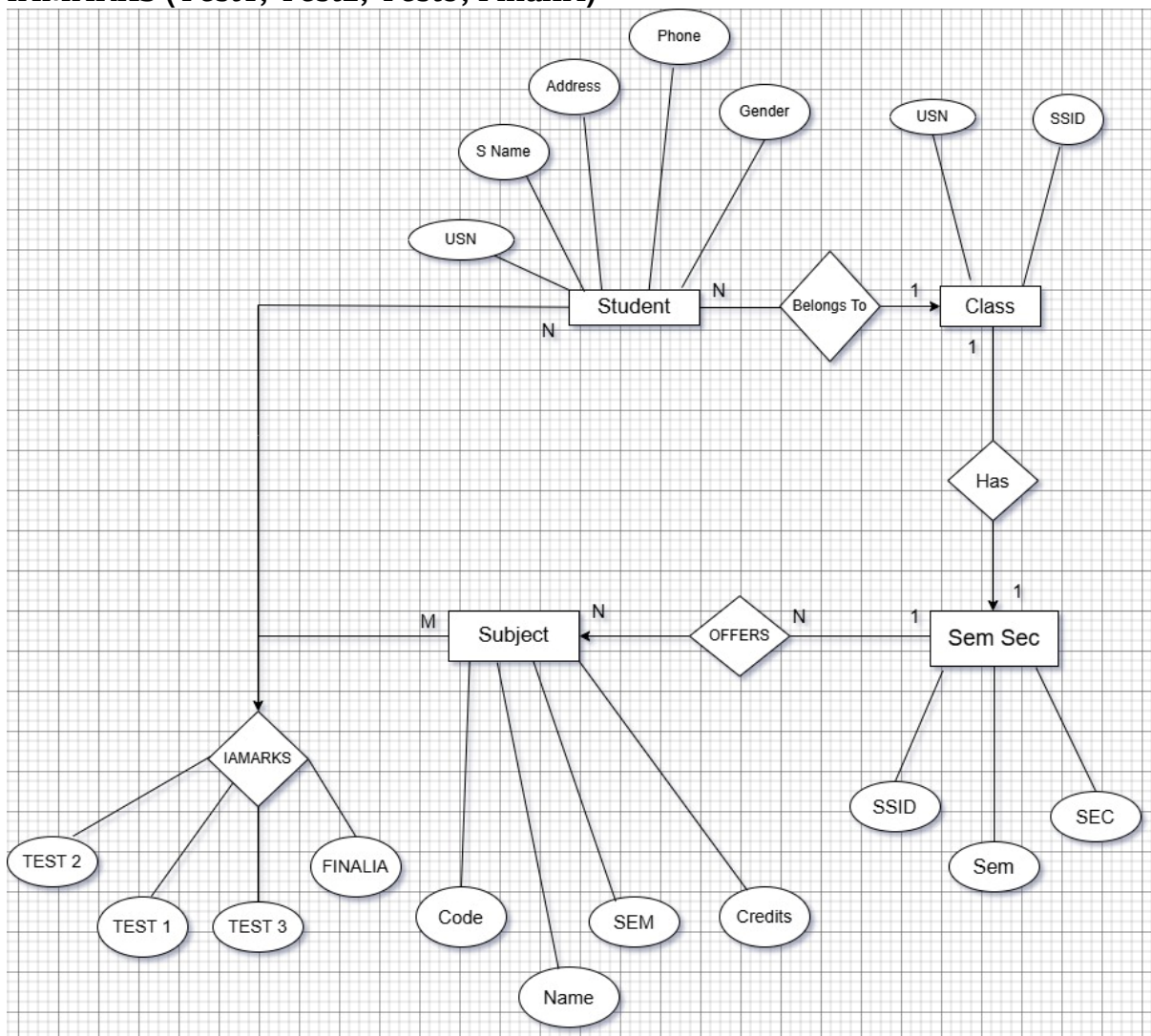
STUDENT (USN, S Name, Address, Phone,

Gender) SEMSEC (SSID, Sem, SEC)

CLASS (USN, SSID)

SUBJECT (Code, Name, Sem, Credits)

IAMARKS (Test1, Test2, Test3, FinalIA)

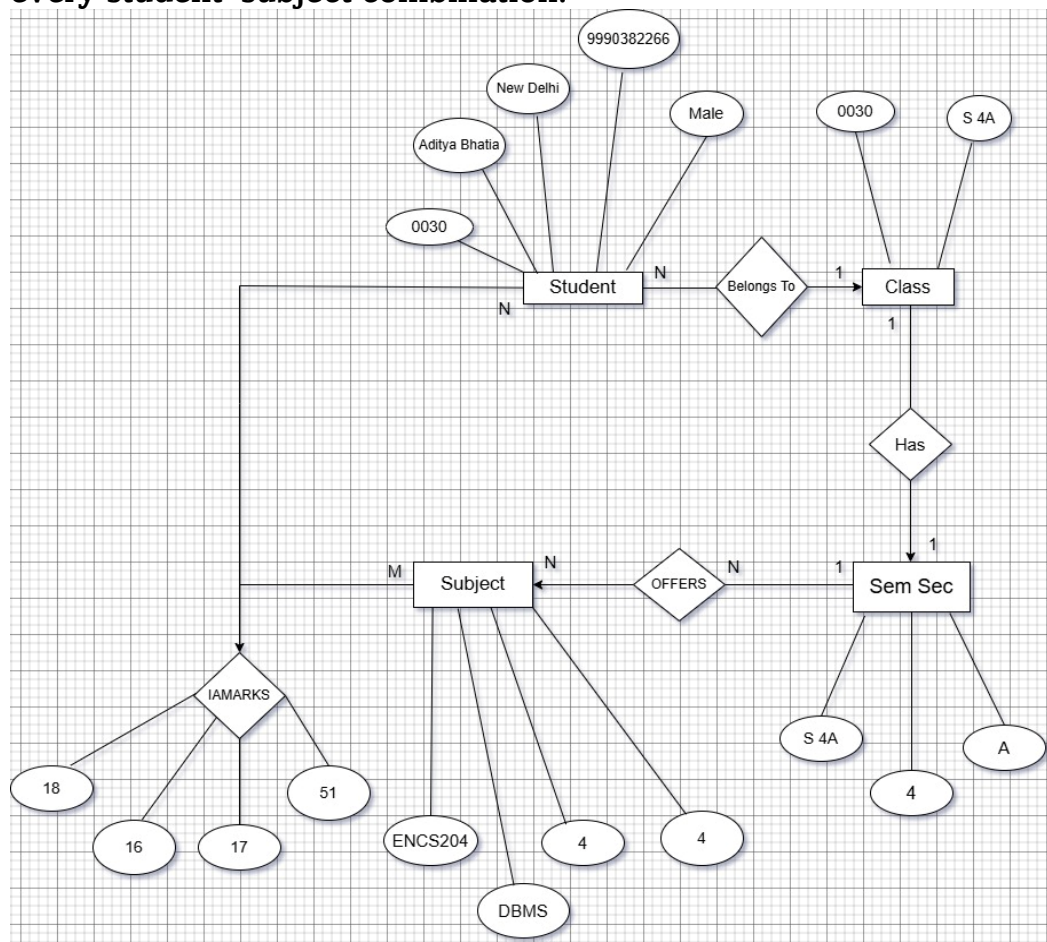


Explanation of ER Diagram – College Database :

- This ER diagram represents a college database that stores information about students, classes, semester–sections, subjects, and internal assessment marks.
- In this diagram, the Student entity stores the basic details of a student such as USN, student name, address, phone number, and gender.

Every student belongs to a particular class. This relationship is shown using the BELONGS TO relationship. Many students can belong to the same class, but a student can belong to only one class.

- The Class entity represents a group of students. Each class is associated with exactly one semester and section, which is represented by the SemSec entity. The relationship between Class and SemSec is one-to-one, meaning each class has only one semester-section and each semester-section belongs to one class.
- The SemSec entity stores information about the semester and section. A semester-section offers multiple subjects. This is shown by the OFFERS relationship between SemSec and Subject. One semester-section can offer many subjects, but each subject is offered in one specific semester-section.
- The Subject entity stores subject details such as subject code, subject name, semester, and credits. Students study multiple subjects, and each subject is studied by many students. This many-to-many relationship is represented using the IAMARKS relationship.
- The IAMARKS relationship stores the internal assessment marks of students for each subject. It includes Test1, Test2, Test3, and Final IA. This relationship connects Student and Subject and stores marks for every student–subject combination.



College ER Diagram Explained Using Real Example :

- Consider a student named Aditya Bhatia whose USN is 0030. He lives in New Delhi, his phone number is 9990382266, and his gender is Male. Aditya belongs to a class named S4A, which is associated with semester 4 and sectionA.
- The Class S4A is linked with one Semester–Section entity that stores the details of semester 4 and section A. This shows that each class is associated with exactly one semester and section.
- In semester 4, several subjects are offered. One of the subjects is DBMS with subject code ENS204, semester 4, and 4 credits. Aditya studies DBMS along with other subjects. This information is represented using the OFFERS relationship between SemSec and Subject.
- Aditya's internal assessment marks in DBMS are Test1 = 18, Test2 = 16, Test3 = 17, and Final IA = 51. These marks are stored in the IAMARKS relationship, which connects the Student entity to the Subject entity and records the marks for each student–subject combination.

Experiment 3

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string) **Item**(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;
USE CustomerSaleDB;
```

```
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price INT NOT NULL CHECK (price > 0)
);
```

```
CREATE TABLE Sale (
    bill_no INT,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT NOT NULL CHECK (qty_sold > 0),
    PRIMARY KEY (bill_no, item_id),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

	cust_id	cust_name
*	NULL	NULL

	item_id	item_name	price
*	NULL	NULL	NULL

	bill_no	bill_date	cust_id	item_id	qty_sold
*	NULL	NULL	NULL	NULL	NULL

2. Insert around 10 records in each of the tables.

	bill_no	cust_name	item_id
▶	1001	Ishaan	101
	1002	Mahin	102
	1003	Aditya	104
	1004	Ayush	105
	1005	Priya	108
	1006	Karan	110
	1007	Neha	103
	1008	Rohit	109
	1009	Simran	106
	1010	Vikas	107

INSERT INTO Customer VALUES

(1,'Ishaan'),
(2,'Mahin'),
(3,'Aditya'),
(4,'Ayush'),
(5,'Priya'),
(6,'Karan'),
(7,'Neha'),
(8,'Rohit'),
(9,'Simran'),
(10,'Vikas');

INSERT INTO Item VALUES

(101,'Keyboard',50000),
(102,'Mouse',500),
(103,'Keyboard',800),
(104,'Monitor',12000),
(105,'USB Cable',200),
(106,'Headphones',1500),
(107,'Charger',900),
(108,'Tablet',25000),
(109,'Smartphone',30000),
(110,'Pen Drive',700);

INSERT INTO Sale VALUES

(1001,CURDATE(),1,101,1),
(1002,CURDATE(),2,102,2),
(1003,CURDATE(),3,104,1),
(1004,CURDATE(),4,105,3),
(1005,CURDATE(),5,108,1),
(1006,CURDATE(),6,110,4),
(1007,CURDATE(),7,103,2),
(1008,CURDATE(),8,109,1),
(1009,CURDATE(),9,106,1),
(1010,CURDATE(),10,107,2);

	cust_id	cust_name		item_id	item_name	price
▶	1	Ishaan	▶	101	Keyboard	50000
	2	Mahin		102	Mouse	500
	3	Aditya		103	Keyboard	800
	4	Ayush		104	Monitor	12000
	5	Priya		105	USB Cable	200
	6	Karan		106	Headphones	1500
	7	Neha		107	Charger	900 900
	8	Rohit		108	Tablet	25000
	9	Simran		109	Smartphone	30000
	10	Vikas		110	Pen Drive	700
•	NULL	NULL	•	NULL	NULL	NULL

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name,
s.item_id FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

	bill_no	bill_date	cust_id	item_id	qty_sold
▶	1001	2026-02-28	1	101	1
	1002	2026-02-28	2	102	2
	1003	2026-02-28	3	104	1
	1004	2026-02-28	4	105	3
	1005	2026-02-28	5	108	1
	1006	2026-02-28	6	110	4
	1007	2026-02-28	7	103	2
	1008	2026-02-28	8	109	1
	1009	2026-02-28	9	106	1
	1010	2026-02-28	10	107	2
•	NULL	NULL	NULL	NULL	NULL

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
    s.bill_no,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

	bill_no	qty_sold	price	final_amount
▶	1001	1	50000	50000
	1002	2	500	1000
	1003	1	12000	12000
	1004	3	200	600
	1005	1	25000	25000
	1006	4	700	2800
	1007	2	800	1600
	1008	1	30000	30000
	1009	1	1500	1500
	1010	2	900	1800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id =
s.cust_id JOIN Item i ON s.item_id
= i.item_id WHERE i.price > 200;
```

	cust_id	cust_name
▶	1	Ishaan
	2	Mahin
	7	Neha
	3	Aditya
	9	Simran
	10	Vikas
	5	Priya
	8	Rohit
	6	Karan

6. Give a count of how many products have been bought by each customer

```
SELECT
    c.cust_id,
    c.cust_name,
    SUM(s.qty_sold) AS
total_products_bought FROM Customer c
JOIN Sale s ON c.cust_id =
s.cust_id GROUP BY c.cust_id,
c.cust_name;
```

	cust_id	cust_name	total_products_bought
▶	1	Ishaan	1
	2	Mahin	2
	3	Aditya	1
	4	Ayush	3
	5	Priya	1
	6	Karan	4
	7	Neha	2
	8	Rohit	1
	9	Simran	1
	10	Vikas	2

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

	item_id	item_name
▶	108	Tablet

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

	item_id	item_name	price
▶	101	Keyboard	50000
	102	Mouse	500
	104	Monitor	12000
	105	USB Cable	200
	108	Tablet	25000
	110	Pen Drive	700
	103	Keyboard	800
	109	Smartphone	30000
	106	Headphones	1500
	107	Charger	900

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```

CREATE OR REPLACE VIEW Bill_View AS
SELECT
    s.bill_no,
    s.bill_date,
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount FROM
Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_View;

```

	bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
▶	1001	2026-02-28	1	101	50000	1	50000
	1002	2026-02-28	2	102	500	2	1000
	1003	2026-02-28	3	104	12000	1	12000
	1004	2026-02-28	4	105	200	3	600
	1005	2026-02-28	5	108	25000	1	25000
	1006	2026-02-28	6	110	700	4	2800
	1007	2026-02-28	7	103	800	2	1600
	1008	2026-02-28	8	109	30000	1	30000
	1009	2026-02-28	9	106	1500	1	1500
	1010	2026-02-28	10	107	900	2	1800

10. Create a view which lists the daily sales date wise for the last one week

```

CREATE OR REPLACE VIEW Weekly_Sales AS
SELECT
    s.bill_date,
    SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;

```

Bill_date:	2026-02-28
Total_sales:	126300

Experiment 4

QUESTION : Assignment 4

Consider the following table: Table: CLASS ID Name

1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,'Rahul');
- COMMIT;
- UPDATE class SET name = 'Abhijeet' WHERE id= '5';
- SAVEPOINT A;
- INSERT INTO class VALUES (6, 'Chris');
- SAVEPOINT B;
- INSERT INTO class VALUES (7, 'Bravo');
- SAVEPOINT C
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

CODE :

```
SET autocommit = 0;
```

```
CREATE TABLE class (  
id INT,  
name VARCHAR(50)  
);
```

```
INSERT INTO class VALUES (1,'Bravo'); INSERT INTO class VALUES (2,'Alex');  
INSERT INTO class VALUES (4,'Cheng');
```

```
INSERT INTO class VALUES (5,'Rahul'); COMMIT;
```

```
UPDATE class SET name='Abhijeet' WHERE id='5'; SAVEPOINT A;  
INSERT INTO class VALUES (6,'Chris'); SAVEPOINT B;  
INSERT INTO class VALUES (7,'Bravo'); SAVEPOINT C;
```

```
SELECT * FROM class; ROLLBACK TO B;  
SELECT * FROM class; ROLLBACK TO A;
```

Table :

Result Grid		
	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris
●	NULL	NULL

Assignment (4.2)

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

CODE :

```
SET autocommit = 0;
```

```
DROP TABLE IF EXISTS ACCOUNT_TXN; CREATE TABLE ACCOUNT_TXN (  
Acc_ID INT PRIMARY KEY,  
Holder_Name VARCHAR(50), Balance INT  
);
```

```
INSERT INTO ACCOUNT_TXN VALUES (101,'Aman',5000),  
(102,'Deepak',7000),  
(103,'Chirag',6000);
```

```
SELECT * FROM ACCOUNT_TXN; START TRANSACTION;  
INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000); SAVEPOINT S1;  
UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;  
UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;  
SAVEPOINT S2;  
DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103; INSERT INTO  
ACCOUNT_TXN VALUES (105,'Eshan',3000); SAVEPOINT S3;
```

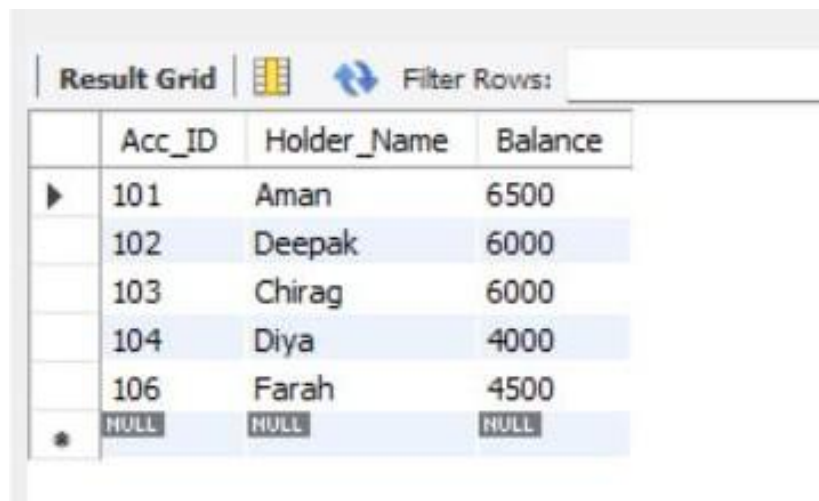
```
UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;  
SELECT * FROM ACCOUNT_TXN;  
ROLLBACK TO S2;
```

```
SELECT * FROM ACCOUNT_TXN;
```

```
INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500); SAVEPOINT S4;  
UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
```

```
SELECT * FROM ACCOUNT_TXN; ROLLBACK TO S4;  
SELECT * FROM ACCOUNT_TXN; COMMIT;
```

OUTPUT :



The screenshot shows a database result grid with the following data:

	Acc_ID	Holder_Name	Balance
▶	101	Aman	6500
	102	Deepak	6000
	103	Chirag	6000
	104	Diya	4000
	106	Farah	4500
•	NULL	NULL	NULL

Experiment 5

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)
CREATE database emp;

use emp;

```
CREATE TABLE EMPLOYEES (  
    Employee_Id INT PRIMARY KEY,  
    First_Name VARCHAR(50),  
    Last_Name VARCHAR(50),  
    Email VARCHAR(100),  
    Phone_Number VARCHAR(20),  
    Hire_Date DATE,  
    Job_Id VARCHAR(20),  
    Salary INT,  
    Commission_Act DECIMAL(5,2),  
    Manager_Id INT,  
    Department_Id INT  
);
```

INSERT INTO EMPLOYEES VALUES

(101, 'John', 'Smith', 'john.smith@email.com', '9876543210', '2020-01-15', 'TT_PROG', 6000, NULL, 100, 60),

(102, 'Sara', 'Connor', 'sara.connor@email.com', '9876543211', '2019-03-10', 'HR_REP', 4500, NULL, 101, 70),

(103, 'Mike', 'Austin', 'mike.austin@email.com', '9876543212', '2021-06-20', 'SA_REP', 5000, 0.10, 100, 80),

(104, 'Emma', 'Watson', 'emma.watson@email.com', '9876543213', '2018-11-05', 'TT_PROG', 7000, NULL, 101, 60),

(105, 'Liam', 'Brown', 'liam.brown@email.com', '9876543214', '2022-02-18', 'SA_REP', 4800, 0.15, 102, 80),

(106, 'Olivia', 'Davis', 'olivia.davis@email.com', '9876543215', '2020-07-25', 'HR_REP', 4200, NULL, 101, 70),

(107, 'Noah', 'Wilson', 'noah.wilson@email.com', '9876543216', '2017-09-30', 'TT_PROG', 8000, NULL, 100, 60),

(108, 'Ava', 'Taylor', 'ava.taylor@email.com', '9876543217', '2019-12-11', 'SA_REP', 4700, 0.12, 102, 80),

(109, 'Ethan', 'Anderson', 'ethan.anderson@email.com', '9876543218', '2021-04-14', 'TT_PROG', 5200, NULL, 101, 60),

(110, 'Sophia', 'Thomas', 'sophia.thomas@email.com', '9876543219', '2018-08-19', 'HR_REP', 4600, NULL, 100, 70),

(111, 'James', 'Jackson', 'james.jackson@email.com', '9876543220', '2020-05-23', 'SA_REP', 4900, 0.20, 102, 80),

(112, 'Isabella', 'White', 'isabella.white@email.com', '9876543221', '2019-01-30', 'IT_PROG', 6100, NULL, 101, 60),

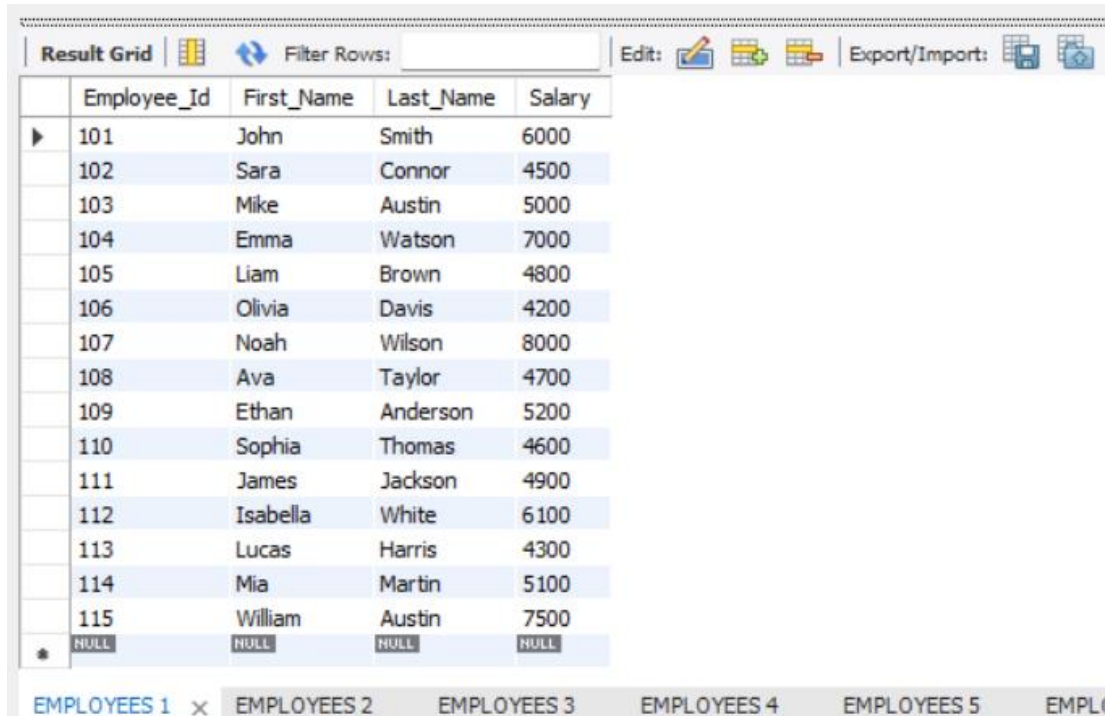
(113, 'Lucas', 'Harris', 'lucas.harris@email.com', '9876543222', '2022-03-12', 'HR_REP', 4300, NULL, 100, 70),

(114, 'Mia', 'Martin', 'mia.martin@email.com', '9876543223', '2021-10-07', 'SA_REP', 5100, 0.18, 102, 80),

(115, 'William', 'Austin', 'william.austin@email.com', '9876543224', '2020-12-01', 'IT_PROG', 7500, NULL, 100, 60);

(a) Find out the employee id, names, salaries of all the employees

```
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```



The screenshot shows a database query result grid with the following columns: Employee_Id, First_Name, Last_Name, and Salary. The data is as follows:

Employee_Id	First_Name	Last_Name	Salary
101	John	Smith	6000
102	Sara	Connor	4500
103	Mike	Austin	5000
104	Emma	Watson	7000
105	Liam	Brown	4800
106	Olivia	Davis	4200
107	Noah	Wilson	8000
108	Ava	Taylor	4700
109	Ethan	Anderson	5200
110	Sophia	Thomas	4600
111	James	Jackson	4900
112	Isabella	White	6100
113	Lucas	Harris	4300
114	Mia	Martin	5100
115	William	Austin	7500
NULL	NULL	NULL	NULL

The interface includes a 'Result Grid' tab, a 'Filter Rows' field, and buttons for 'Edit', 'Export/Import', and 'EMPL'.

(b) List out the employees who works under manager 100

```
SELECT *
FROM EMPLOYEES
WHERE Manager_Id = 100;
```

Result Grid											
Filter Rows:											
Edit: Export/Import: Wrap Cell Content: TA											
	Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
▶	101	John	Smith	john.smith@email.com	9876543210	2020-01-15	IT_PROG	6000	NULL	100	60
	103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
	107	Noah	Wilson	noah.wilson@email.com	9876543216	2017-09-30	IT_PROG	8000	NULL	100	60
	110	Sophia	Thomas	sophia.thomas@email.com	9876543219	2018-08-19	HR_REP	4600	NULL	100	70
	113	Lucas	Harris	lucas.harris@email.com	9876543222	2022-03-12	HR_REP	4300	NULL	100	70
	115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1 EMPLOYEES 2 x EMPLOYEES 3 EMPLOYEES 4 EMPLOYEES 5 EMPLOYEES 6 Apply

(c) Find the names of the employees who have a salary greater than or equal to 4800
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Salary >= 4800;

Result Grid		
Filter Rows:		
Export: Wrap C		
	First_Name	Last_Name
▶	John	Smith
	Mike	Austin
	Emma	Watson
	Liam	Brown
	Noah	Wilson
	Ethan	Anderson
	James	Jackson
	Isabella	White
	Mia	Martin
	William	Austin

EMPLOYEES 1 EMPLOYEES 2 EMPLOYEES 3 x EMPLOYEES 4

(d) List out the employees whose last name is 'AUSTIN'
SELECT *
FROM EMPLOYEES
WHERE Last_Name = 'AUSTIN';

Result Grid										
Filter Rows:										
Edit: Export/Import: Wrap Cell Content:										
Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1	EMPLOYEES 2	EMPLOYEES 3	EMPLOYEES 4 x	EMPLOYEES 5	EMPLOYEES 6	Apply
-------------	-------------	-------------	---------------	-------------	-------------	-------

(e) Find the names of the employees who works in departments 60,70 and 80

SELECT First_Name, Last_Name

FROM EMPLOYEES

WHERE Department_Id IN (60, 70, 80);

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	First_Name	Last_Name
▶	John	Smith
	Sara	Connor
	Mike	Austin
	Emma	Watson
	Liam	Brown
	Olivia	Davis
	Noah	Wilson
	Ava	Taylor
	Ethan	Anderson
	Sophia	Thomas
	James	Jackson
	Isabella	White
	Lucas	Harris
	Mia	Martin
	William	Austin

EMPLOYEES 1

EMPLOYEES 2

EMPLOYEES 3

EMPLOYEES 4

EMPLOYEES 5

×

EMPLOYEES 1 EMPLOYEES 2 EMPLOYEES 3 EMPLOYEES 4 EMPLOYEES 5 x

(f) Display the unique Manager_Id
SELECT DISTINCT Manager_Id
FROM EMPLOYEES;

Result Grid	
Filter Rows:	
	Manager_Id
▶	100
	101
	102

Experiment 6

Assignment 6

(Exercise on updating records in table) Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due);

Create database Client_master;

use Client_master;

```
CREATE TABLE Client_master (  
    ClientNO VARCHAR(10) PRIMARY KEY,  
    Name VARCHAR(50),  
    Address VARCHAR(100),  
    City VARCHAR(50),  
    State VARCHAR(50),  
    bal_due DECIMAL(10,2)  
);
```

(a) Insert five records

INSERT INTO Client_master VALUES

('C106', 'Anjali Kapoor', 'Sector 18', 'Noida', 'Uttar Pradesh', 4800),

('C107', 'Vikas Malhotra', 'Connaught Place', 'Delhi', 'Delhi', 8000),

('C108', 'Sneha Reddy', 'Banjara Hills', 'Hyderabad', 'Telangana', 6200),

('C109', 'Arjun Nair', 'Marine Drive', 'Mumbai', 'Maharashtra', 5300),

('C110', 'Karan Patel', 'SG Highway', 'Ahmedabad', 'Gujarat', 4500);

Result Grid

Filter Rows:

Edit:

Export/Import:

	ClientNO	Name	Address	City	State	bal_due
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
★	NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3 x Client_master 4 Client_master 5 Client12 6

(b) Find the names of clients whose bal_due > 5000 .

SELECT Name

FROM Client_master

WHERE bal_due > 5000;

Result Grid		Filter Rows:
	Name	
▶	Vikas Malhotra	
	Sneha Reddy	
	Arjun Nair	

Client_master 3 Client_master 4 × Cl

(c) Change the bal_due of ClientNO “ C123” to Rs. 5100

```
UPDATE Client_master
SET bal_due = 5100
WHERE ClientNO = 'C123';
```

Result Grid

Filter Rows:

Edit:

Export/Import:

	ClientNO	Name	Address	City	State	bal_due
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3

Client_master 4

Client_master 5 ×

Client12 6

(d) Change the name of Client_master to Client12.

```
ALTER TABLE Client_master
RENAME TO Client12;
```

(e) Display the bal_due heading as “BALANCE”

```
SELECT ClientNO, Name, Address, City, State, bal_due AS BALANCE
FROM Client12;
```

Result Grid Filter Rows: Edit: Export/Import:						
	ClientNO	Name	Address	City	State	BALANCE
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3

Client_master 4

Client_master 5

Client12 6 ×

Experiment 7

Objective: Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

- a) Insert five records.
- b) Give Increment of 25% salary for Mathematics Department.
- c) Perform Rollback command.
- d) Give increment of 15% salary for Commerce Department.
- e) Perform Commit command.

SOLUTION CODE:

```
-- Create and use database
CREATE DATABASE IF NOT EXISTS institute_db;
USE institute_db;
```

```
-- Disable safe updates
SET SQL_SAFE_UPDATES = 0;
```

```
-- Drop table if exists
DROP TABLE IF EXISTS Faculty;
```

```
-- Create table
CREATE TABLE Faculty (
    FacultyName VARCHAR(50),
    DepartmentNo INT,
    JoiningDate DATE,
    DepartmentName VARCHAR(50),
    City VARCHAR(50),
    Salary INT
);
```

(a) Insert five records.

```
INSERT INTO Faculty (FacultyName, DepartmentNo, JoiningDate,
DepartmentName, City, Salary) VALUES
('Arjun Mehta', 11, '2022-02-12', 'Mathematics', 'Delhi', 45000),
('Kavya Singh', 22, '2021-06-18', 'Commerce', 'Mumbai', 38000),
('Rohan Verma', 11, '2023-01-05', 'Mathematics', 'Pune', 47000),
('Sneha Iyer', 33, '2020-09-25', 'Science', 'Chennai', 52000),
('Vikas Gupta', 22, '2022-11-30', 'Commerce', 'Jaipur', 36000);
```

(b) Give Increment of 25% salary for Mathematics Department.

```
START TRANSACTION;
UPDATE Faculty
SET Salary = Salary * 1.25
WHERE DepartmentName = 'Mathematics';
SELECT * FROM Faculty WHERE DepartmentName = 'Mathematics';
```

Result Grid Filter Rows: Export: Wrap Cell Content:						
	FacultyName	DepartmentNo	JoiningDate	DepartmentName	City	Salary
▶	Arjun Mehta	11	2022-02-12	Mathematics	Delhi	56250
	Rohan Verma	11	2023-01-05	Mathematics	Pune	58750

(c) Perform Rollback command.

ROLLBACK;

SELECT * FROM Faculty WHERE DepartmentName = 'Mathematics';

Result Grid Filter Rows: Export: Wrap Cell Content:						
	FacultyName	DepartmentNo	JoiningDate	DepartmentName	City	Salary
▶	Arjun Mehta	11	2022-02-12	Mathematics	Delhi	45000
	Rohan Verma	11	2023-01-05	Mathematics	Pune	47000

Faculty 1
Faculty 2 ×
Faculty 3

(d) Give increment of 15% salary for Commerce Department.

START TRANSACTION;

UPDATE Faculty

SET Salary = Salary * 1.15

WHERE DepartmentName = 'Commerce';

(e) Perform Commit command.

COMMIT;

SELECT * FROM Faculty;

SET SQL_SAFE_UPDATES = 1;

Result Grid Filter Rows: Export: Wrap Cell Content:						
	FacultyName	DepartmentNo	JoiningDate	DepartmentName	City	Salary
▶	Arjun Mehta	11	2022-02-12	Mathematics	Delhi	45000
	Kavya Singh	22	2021-06-18	Commerce	Mumbai	43700
	Rohan Verma	11	2023-01-05	Mathematics	Pune	47000
	Sneha Iyer	33	2020-09-25	Science	Chennai	52000
	Vikas Gupta	22	2022-11-30	Commerce	Jaipur	41400

Experiment 8

AIM: Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

f) Insert five records.

g) Calculate total salesamount in each branch.

h) Calculate average salesamount in each branch.

i) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.

j) Display the name and DOB of salesman in alphabetical order of the month.

CODE:

```
CREATE DATABASE IF NOT EXISTS retail_sales_db;
```

```
USE retail_sales_db;
```

```
-- Drop table if exists
```

```
DROP TABLE IF EXISTS SalesTeam;
```

```
-- Create table
```

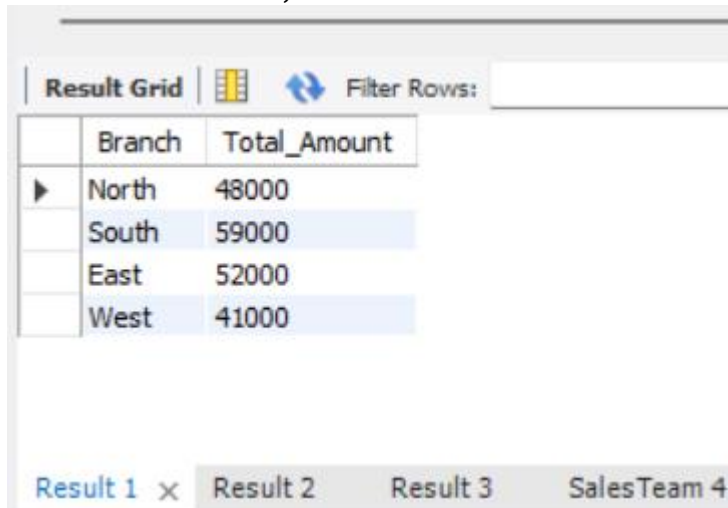
```
CREATE TABLE SalesTeam (  
    SaleID INT PRIMARY KEY,  
    SalesPerson VARCHAR(50),  
    Branch VARCHAR(50),  
    Amount INT,  
    DOB DATE  
);
```

(a) Insert five records.

```
INSERT INTO SalesTeam (SaleID, SalesPerson, Branch, Amount, DOB) VALUES  
(201, 'Aditya', 'North', 48000, '1997-12-21'),  
(202, 'Ishaan', 'South', 30000, '1998-04-10'),  
(203, 'Mahin', 'East', 52000, '1996-12-09'),  
(204, 'Diya', 'West', 41000, '1999-12-05'),  
(205, 'Ishika', 'South', 29000, '2000-07-18');
```

(b) Calculate total sales amount in each branch.

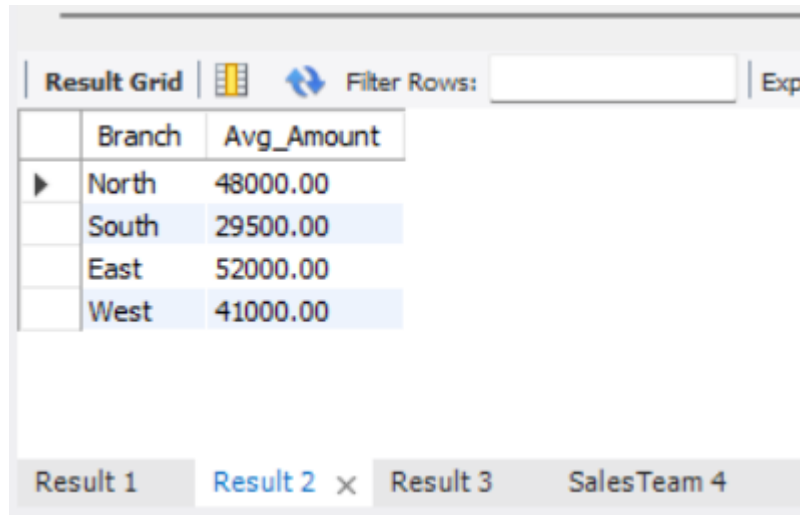
```
SELECT Branch, SUM(Amount) AS Total_Amount  
FROM SalesTeam  
GROUP BY Branch;
```



	Branch	Total_Amount
▶	North	48000
	South	59000
	East	52000
	West	41000

(c) Calculate average sales amount in each branch.

```
SELECT Branch, ROUND(AVG(Amount), 2) AS Avg_Amount  
FROM SalesTeam  
GROUP BY Branch;
```



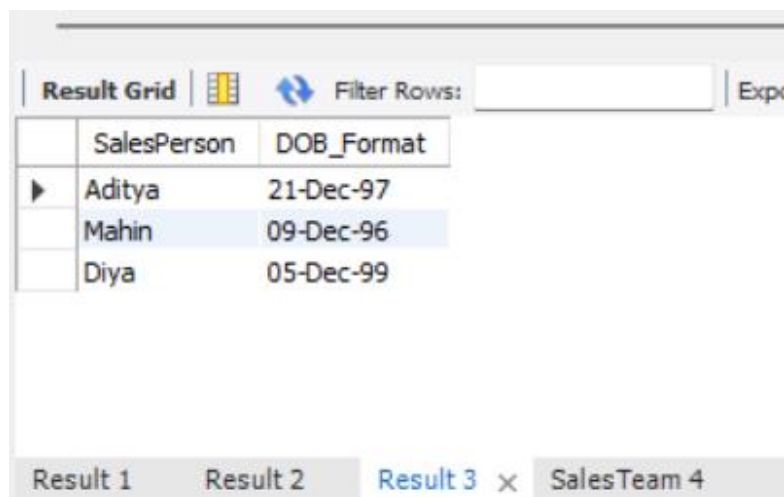
The screenshot shows a SQL query result grid with the following data:

	Branch	Avg_Amount
▶	North	48000.00
	South	29500.00
	East	52000.00
	West	41000.00

At the bottom of the grid, there are tabs for 'Result 1', 'Result 2' (which is active and has a close button), 'Result 3', and 'SalesTeam 4'.

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.

```
SELECT SalesPerson, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB_Format  
FROM SalesTeam  
WHERE MONTH(DOB) = 12;
```



The screenshot shows a SQL query result grid with the following data:

	SalesPerson	DOB_Format
▶	Aditya	21-Dec-97
	Mahin	09-Dec-96
	Diya	05-Dec-99

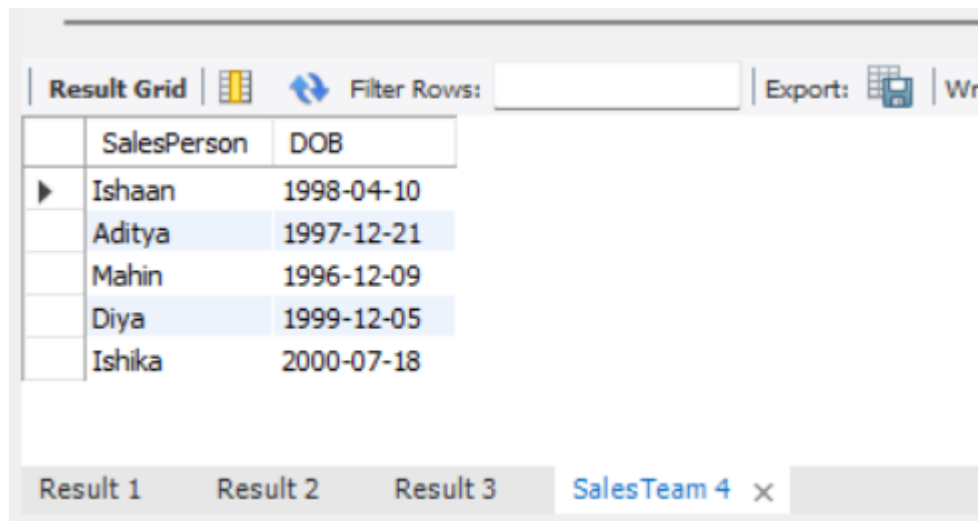
At the bottom of the grid, there are tabs for 'Result 1', 'Result 2', 'Result 3' (which is active and has a close button), and 'SalesTeam 4'.

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
SELECT SalesPerson, DOB
```

```
FROM SalesTeam
```

```
ORDER BY MONTHNAME(DOB);
```



The screenshot shows a SQL query result grid with the following data:

	SalesPerson	DOB
▶	Ishaan	1998-04-10
	Aditya	1997-12-21
	Mahin	1996-12-09
	Diya	1999-12-05
	Ishika	2000-07-18

The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and an 'Export' button. The bottom of the window shows tabs for 'Result 1', 'Result 2', 'Result 3', and 'SalesTeam 4'.

Experiment 9

AIM: Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES”
Their schemas are as follows:

Departments (dept no , dept name , dept_location);

Employees (emp_id , emp_name , emp_salary,dept_no);

- k) Develop a query to grant all privileges of employees table into departments table.
- l) Develop a query to grant some privileges of employees table into departments table.
- m) Develop a query to revoke all privileges of employees table from departments table.
- n) Develop query to revoke some privileges of employees table from departments table.
- o) Write a query to implement the save point.

CODE:

```
CREATE DATABASE IF NOT EXISTS org_data;
USE org_data;
-- Create Department Table
CREATE TABLE IF NOT EXISTS Dept (
    Dept_ID INT PRIMARY KEY,
    Dept_Title VARCHAR(50),
    Location VARCHAR(50)
);
-- Create Staff Table
CREATE TABLE IF NOT EXISTS Staff (
    Staff_ID INT PRIMARY KEY,
    Staff_Name VARCHAR(50),
    Salary INT,
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Dept(Dept_ID)
);
-- Create User
CREATE USER IF NOT EXISTS 'manager_user'@'localhost' IDENTIFIED BY 'pass@123';
```

(a) Develop a query to grant all privileges of employees table into departments table.

```
GRANT ALL PRIVILEGES ON org_data.Staff TO 'manager_user'@'localhost';
```

(b) Develop a query to grant some privileges of employees table into departments table.

```
GRANT SELECT, INSERT ON org_data.Staff TO 'manager_user'@'localhost';
```

(c) Develop a query to revoke all privileges of employees table from departments table.

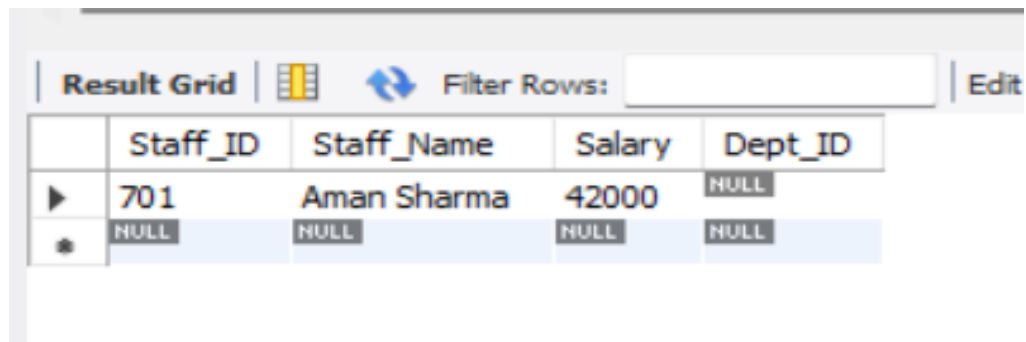
```
REVOKE SELECT, INSERT, UPDATE, DELETE ON org_data.Staff FROM
'manager_user'@'localhost';
```

(d) Develop a query to revoke some privileges of employees table from departments table.

```
REVOKE INSERT ON org_data.Staff FROM 'manager_user'@'localhost';
```

(e) Write a query to implement the save point.

```
START TRANSACTION;  
INSERT INTO Staff (Staff_ID, Staff_Name, Salary, Dept_ID)  
VALUES (701, 'Aman Sharma', 40000, NULL);  
SAVEPOINT sp_one;  
UPDATE Staff  
SET Salary = 42000  
WHERE Staff_ID = 701;  
SELECT * FROM Staff;  
COMMIT;  
SELECT * FROM Staff;
```



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of a query, showing two rows of data from the Staff table. The first row has Staff_ID 701, Staff_Name 'Aman Sharma', Salary 42000, and Dept_ID NULL. The second row has all NULL values. The interface includes a 'Filter Rows' search bar and an 'Edit' button.

	Staff_ID	Staff_Name	Salary	Dept_ID
▶	701	Aman Sharma	42000	NULL
★	NULL	NULL	NULL	NULL

Experiment 9

AIM: Using the tables “DEPARTMENTS” and “EMPLOYEES” from Lab 9 perform the following queries:

- a) Display the employee details, departments that the departments are same in both the emp and dept.
- b) Display the employee name and Department name by implementing a left outer join.
- c) Display the employee name and Department name by implementing a right outer join.
- d) Display the details of those who draw the salary greater than the average salary.

CODE:

```
CREATE DATABASE IF NOT EXISTS student_org;
USE student_org;
CREATE TABLE IF NOT EXISTS CourseDept (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50)
);
CREATE TABLE IF NOT EXISTS Student (
    Stu_ID INT PRIMARY KEY,
    Stu_Name VARCHAR(50),
    Subject VARCHAR(50),
    Salary INT,
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES CourseDept(Dept_ID)
);
INSERT INTO CourseDept VALUES
(1, 'Python'),
(2, 'C++'),
(3, 'Java'),
(4, 'SQL'),
(5, 'C-Sharp');
INSERT INTO Student VALUES
(101, 'Aditya', 'Python', 50000, 1),
```

(102, 'Ishika', 'Java', 60000, 3),
 (103, 'Diya', 'SQL', 45000, 4),
 (104, 'Mahin', 'C++', 70000, 2),
 (105, 'Ishaan', 'C-Sharp', 55000, NULL);

(a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT s.*, d.Dept_Name
```

```
FROM Student s
```

```
INNER JOIN CourseDept d
```

```
ON s.Dept_ID = d.Dept_ID;
```

The screenshot shows a database query result grid with the following data:

	Stu_ID	Stu_Name	Subject	Salary	Dept_ID	Dept_Name
▶	101	Aditya	Python	50000	1	Python
	102	Ishika	Java	60000	3	Java
	103	Diya	SQL	45000	4	SQL
	104	Mahin	C++	70000	2	C++

At the bottom of the grid, there are tabs labeled 'Result 1', 'Result 2', 'Result 3', and 'Student 4'. 'Result 1' is currently selected.

(b) Display the employee name and Department name by implementing a left outer join.

```
SELECT s.Stu_Name, d.Dept_Name
```

```
FROM Student s
```

```
LEFT JOIN CourseDept d
```

```
ON s.Dept_ID = d.Dept_ID;
```

The screenshot shows a database query result grid with the following data:

	Stu_Name	Dept_Name
▶	Aditya	Python
	Ishika	Java
	Diya	SQL
	Mahin	C++
	Ishaan	NULL

At the bottom of the grid, there are tabs labeled 'Result 1', 'Result 2', 'Result 3', and 'Student 4'. 'Result 2' is currently selected.

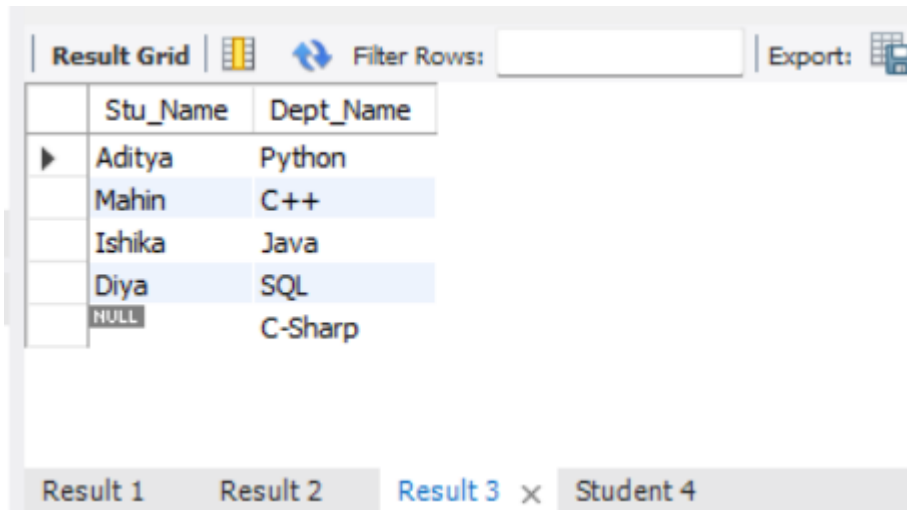
(c) Display the employee name and Department name by implementing a right outer join.

```
SELECT s.Stu_Name, d.Dept_Name
```

```
FROM Student s
```

```
RIGHT JOIN CourseDept d
```

```
ON s.Dept_ID = d.Dept_ID;
```



The screenshot shows a SQL query result grid with the following data:

Stu_Name	Dept_Name
Aditya	Python
Mahin	C++
Ishika	Java
Diya	SQL
NULL	C-Sharp

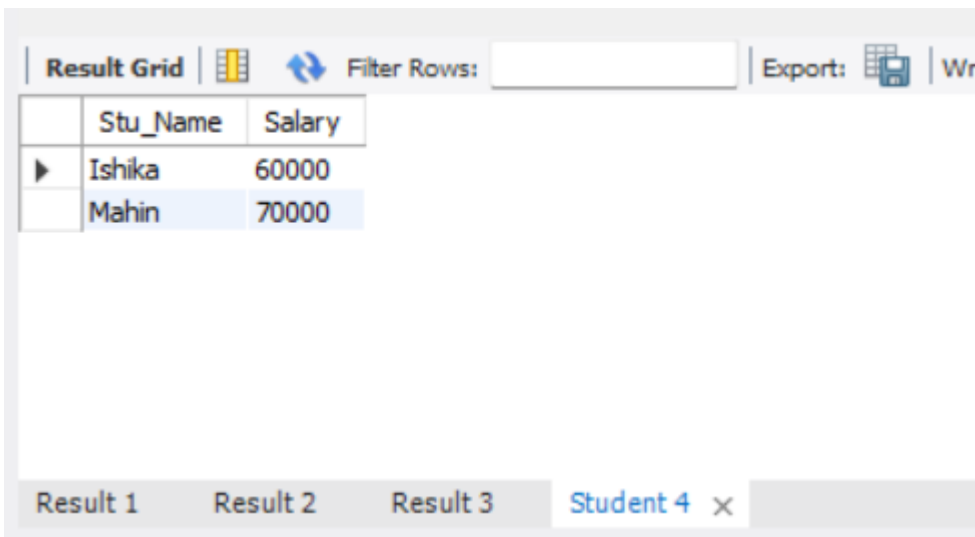
The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and an 'Export' button. The bottom of the window shows tabs for 'Result 1', 'Result 2', 'Result 3', and 'Student 4'.

(d) Display the details of those who draw the salary greater than the average salary.

```
SELECT Stu_Name, Salary
```

```
FROM Student
```

```
WHERE Salary > (SELECT AVG(Salary) FROM Student);
```



The screenshot shows a SQL query result grid with the following data:

Stu_Name	Salary
Ishika	60000
Mahin	70000

The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and an 'Export' button. The bottom of the window shows tabs for 'Result 1', 'Result 2', 'Result 3', and 'Student 4'.

Experiment 10

AIM: Using the tables “DEPARTMENTS” and “EMPLOYEES” from Lab 9 perform the following queries:

- Display the employee details, departments that the departments are same in both the emp and dept.
- Display the employee name and Department name by implementing a left outer join.
- Display the employee name and Department name by implementing a right outer join.
- Display the details of those who draw the salary greater than the average salary.

CODE:

```
CREATE DATABASE IF NOT EXISTS company_lab;
USE company_lab;
CREATE TABLE IF NOT EXISTS Departments (
    DeptNo INT PRIMARY KEY,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);
CREATE TABLE IF NOT EXISTS Employees (
    EmpId INT PRIMARY KEY,
    EmpName VARCHAR(50),
    EmpSalary INT,
    DeptNo INT,
    FOREIGN KEY (DeptNo) REFERENCES Departments(DeptNo)
);
INSERT INTO Departments VALUES
(11, 'Python', 'Block A'),
(22, 'C++', 'Block B'),
(33, 'Java', 'Block C'),
(44, 'SQL', 'Block D'),
(55, 'C-Sharp', 'Block E');

INSERT INTO Employees VALUES
(901, 'Aditya', 52000, 11),
(902, 'Ishika', 61000, 33),
(903, 'Diya', 43000, 44),
(904, 'Mahin', 72000, 22),
(905, 'Ishaan', 50000, NULL);
```

(a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT E.*, D.DeptName, D.Location
FROM Employees E
INNER JOIN Departments D
ON E.DeptNo = D.DeptNo;
```

Result Grid						
Filter Rows:						
	EmpId	EmpName	EmpSalary	DeptNo	DeptName	Location
▶	901	Aditya	52000	11	Python	Block A
	904	Mahin	72000	22	C++	Block B
	902	Ishika	61000	33	Java	Block C
	903	Diya	43000	44	SQL	Block D

Result 5 × Result 6 Result 7 Employees 8

(b) Display the employee name and Department name by implementing a left outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptNo = D.DeptNo;
```

Result Grid		
Filter Rows:		
	EmpName	DeptName
▶	Aditya	Python
	Ishika	Java
	Diya	SQL
	Mahin	C++
	Ishaan	NULL

Result 5 Result 6 × Result 7 Employees 8

(c) Display the employee name and Department name by implementing a right outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D
ON E.DeptNo = D.DeptNo;
```

Result Grid		
Filter Rows:		
	EmpName	DeptName
▶	Aditya	Python
	Mahin	C++
	Ishika	Java
	Diya	SQL
	NULL	C-Sharp

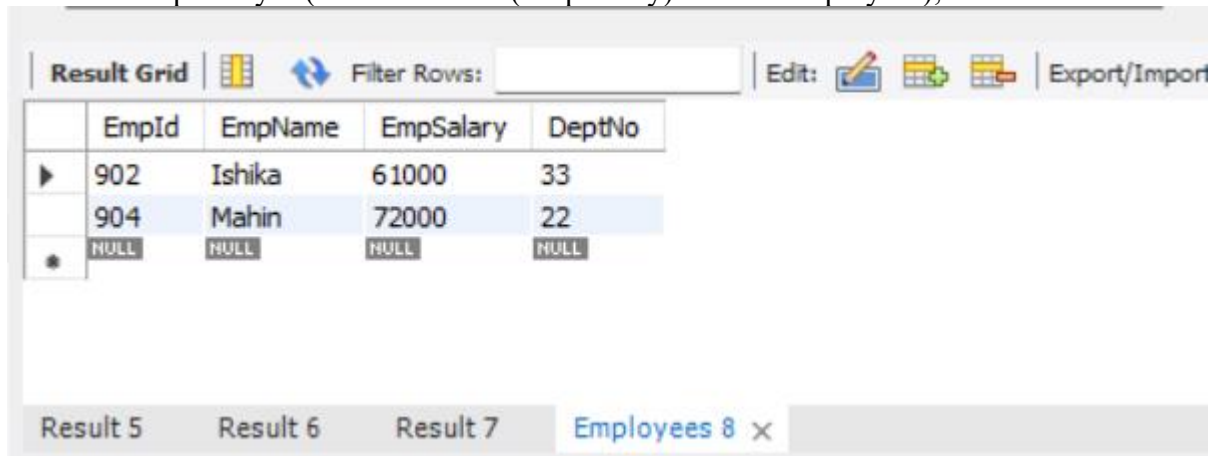
Result 5 Result 6 Result 7 × Employees 8

(d) Display the details of those who draw the salary greater than the average salary.

SELECT *

FROM Employees

WHERE EmpSalary > (SELECT AVG(EmpSalary) FROM Employees);



The screenshot shows a SQL query result grid. The grid has a toolbar at the top with options like 'Result Grid', 'Filter Rows', 'Edit', and 'Export/Import'. The main area displays a table with the following data:

	EmpId	EmpName	EmpSalary	DeptNo
▶	902	Ishika	61000	33
	904	Mahin	72000	22
•	NULL	NULL	NULL	NULL

At the bottom, there is a tab bar with tabs labeled 'Result 5', 'Result 6', 'Result 7', and 'Employees 8 x'.